LAB 1 — How to Store an AI Dataset?

Data Storage Methods in AI Systems

1. Laboratory Objective

The aim of this laboratory is to explore and practically compare different methods of storing data used in Artificial Intelligence systems.

During the laboratory, you will analyze how different components of an AI system can be stored, compare popular data formats, and investigate how the selected storage method affects file size and data processing performance.

The laboratory is not limited to writing code. Some tasks require you to analyze the problem, propose a solution, and justify your decisions.

2. Submission Format

The laboratory should be completed individually or in a group of at most 2 persons.

Submit your work in one of the following formats:

- a Notebook file (*.ipynb) containing the code, results, explanations, and answers,

or

- a report in *.pdf format containing the most important code fragments, experiment results, tables, charts, and answers.

Use the same task numbering as in this instruction.

3. Environment Preparation

The following Python libraries will be used:

```
pandas
numpy
pyarrow
duckdb
matplotlib
pathlib
time
os
```

If necessary, install the required packages:

Task 1. Design the Storage Architecture of an AI System

Consider an AI system for image classification. The dataset contains 100,000 images. For each image, the following metadata is stored:

```
image_id
filename
category
description
width
height
source
created_at
```

The AI system also uses:

- a train, validation, and test split,
- model predictions,
- class probabilities,
- a 512-dimensional embedding for each image,
- information about the model version,
- information about training experiments.

Design a storage solution for such a system. Consider the following elements separately:

```
images
image metadata
train/validation/test split
model predictions
embeddings
model information
experiment information
```

For each element, specify:

1. where or in which format it should be stored,
2. why this solution was selected,
3. what limitations the solution may have,
4. whether the same solution would still be reasonable if the dataset increased from 100,000 to 10,000,000 images.

There is no single correct answer. The quality of your justification is important.

What to include in the report

- a table presenting the proposed storage method for each component,
- a simple diagram of the proposed data architecture,
- justification of the most important design decisions,
- a short answer: Which part of the system would become the largest storage or performance problem if the dataset grew to 10 million images, and why?

Task 2. Prepare Data for the Experiment

Create a DataFrame representing metadata of an AI dataset. The DataFrame should contain at least 100,000 records and the following columns:

```
image_id
filename
category
width
height
source
split
confidence
model_version
```

Example categories:

```
cat
dog
car
person
other
```

Example split values:

```
train
validation
test
```

Example model versions:

```
v1
v2
v3
```

The data may be generated automatically. Try to make the generated dataset reasonably realistic:

- the classes do not need to contain the same number of samples,
- width and height should correspond to realistic image resolutions,
- confidence should be between 0 and 1,
- the training set should be larger than the validation and test sets.

What to include in the report

- the code used to generate or prepare the data,
- the output of df.head(),
- the number of rows and columns,
- data types using df.dtypes,
- the number of records per category,
- the number of records in train, validation, and test,
- a short description of the generated dataset.

Task 3. CSV vs JSON vs Parquet

Save the same DataFrame using three formats: CSV, JSON, and Parquet.

For each format, measure:

5. write time,
6. file size,
7. time required to read the entire dataset.

You may use:

```
from time import perf_counter
```

Example:

```
start = perf_counter()

# operation being measured

elapsed = perf_counter() - start
```

Perform each timing experiment at least 5 times and calculate the average value. Do not base your conclusions on a single measurement.

Format	File size [MB]	Average write time [s]	Average read time [s]
CSV			
JSON			
Parquet			

Next, calculate by what percentage the Parquet file is smaller or larger than the CSV file.

What to include in the report

- the experiment code,
- the comparison table,
- a chart comparing file sizes,
- a chart comparing read times,
- information about whether repeated timing measurements differed significantly,
- at least three specific conclusions based on your results.

Task 4. Does the Storage Format Affect Data Analysis?

Load the data and perform the following operations.

A. Filtering by one class

Find all records where:

```
category == "cat"
```

B. Complex filtering

Find all records satisfying:

```
category == "cat"
confidence > 0.9
```

```
    split == "test"
```

C. Aggregation

For each category, calculate:

- number of images,
- average confidence,
- minimum confidence,
- maximum confidence.

D. Model version analysis

For each model_version, calculate the average confidence.

Compare the execution time for data stored as CSV, JSON, and Parquet.

Think carefully about what is actually being measured. If the entire file is first loaded into a Pandas DataFrame and filtering is performed only afterwards, the original file format may no longer directly affect the filtering operation itself.

What to include in the report

- the code,
- results of all four analyses,
- a timing comparison table,

Task 5. Parquet + DuckDB — Querying Data Directly

In this task, you will use **DuckDB** to query data stored in a Parquet file directly, without first loading the entire dataset into a Pandas DataFrame.

You don't need advanced SQL knowledge to complete this task. Use the examples below as a reference and modify them when necessary.

---

1. Basic SQL Examples

Filtering records

```
SELECT *
FROM 'dataset.parquet'
WHERE category = 'cat';
```

Counting records

```
SELECT category, COUNT(*)
FROM 'dataset.parquet'
GROUP BY category;
```

Calculating an average

```
SELECT category, AVG(confidence)
```

```
FROM 'dataset.parquet'
GROUP BY category;
```

Sorting and selecting a limited number of rows

```
SELECT *
FROM 'dataset.parquet'
ORDER BY confidence ASC
LIMIT 10;
```

---

2. Queries to Perform

Query 1 — Filtering

Find all images satisfying the following conditions:

```
category = 'dog'
confidence > 0.95
split = 'test'
```

Query 2 — Counting Records

Calculate the number of records for each category.

The result should contain:

```
category
number_of_records
```

Query 3 — Average Confidence

Calculate the average `confidence` value for each category.

The result should contain:

```
category
average_confidence
```

Query 4 — Finding the Lowest-Confidence Predictions

Find the **10 records with the lowest `confidence` value**.

Display at least the following columns:

```
image_id
category
confidence
model_version
```

Sort the results from the lowest to the highest confidence value.

Query 5 — Best Category and Model Version Combination

Find the combination of:

```
category
model_version
```

with the highest average `confidence`.

To solve this task, you will need to:

1. group the data by both `category` and `model_version`,
2. calculate the average `confidence`,
3. sort the results from the highest to the lowest average,
4. select the first result.

---

3. Compare DuckDB with Pandas

Choose at least **two queries** from the previous section and perform equivalent operations using Pandas.

For example, an operation such as:

```
SELECT *
FROM 'dataset.parquet'
WHERE category = 'cat';
```

can be compared with an equivalent Pandas operation.

Measure the execution time of both approaches.

Perform each measurement at least **5 times** and calculate the average execution time.

Prepare a table similar to:

Operation	Pandas [s]	DuckDB [s]
Filtering		
Aggregation		

Do not assume that DuckDB will always be faster. Your task is to **measure the performance and interpret the result**.

---

4. Think About What Is Actually Happening

In Pandas, a common workflow is:

```
df = pd.read_parquet("dataset.parquet")
```

followed by operations performed on the DataFrame.

DuckDB can execute a query directly on the Parquet file:

```
SELECT *
FROM 'dataset.parquet'
WHERE category = 'cat';
```

Think about the difference between these two approaches.

Answer the following questions:

1. Does DuckDB need to load the entire dataset before executing every query?
2. Why may reading only the required rows and columns be useful for large datasets?
3. In what situations would you prefer Pandas?
4. In what situations could DuckDB be a better choice?

What to Include in the Report

Include:

- all SQL queries used in the task,
- the results of Queries 1–5,
- Pandas code for at least two equivalent operations,
- an execution-time comparison table,
- the average time calculated from at least 5 measurements,
- answers to the four questions from Section 4.

Task 6. I Do Not Need the Entire Dataset — Column Pruning

Assume that an AI system needs to generate a report containing only:

```
image_id
category
confidence
```

All remaining columns are unnecessary. Compare two approaches.

Approach A

Load the entire file and then select the three required columns.

Approach B

Read only the required columns from the Parquet file.

Measure the execution time of both approaches.

Next, extend your DataFrame so that it contains at least 20 columns, save it again as Parquet, and repeat the experiment.

What to include in the report

- the code,
- a table comparing both approaches,
- information about whether the difference became more visible after increasing the number of columns,
- an explanation of the term column pruning in your own words,
- one example of how column pruning could be useful in an AI system.